02-02-EDA-homework

January 12, 2026

```
<div style='float:left;'><h1>CPSC 4300/6300: Applied Data Science</h1></div>
<img style="float: right; padding-right: 10px; width: 65px" src="https://raw.githubusercontent
```

0.1 Week 2 | Homework : EDA and Visualization

Clemson University Instructor(s): Tim Ransom

0.2 Learning goals

- Identify trends and patterns in data using visualization techniques.
- Interpret descriptive statistics for a dataset.
- Create effective data visualizations using matplotlib.
- Evaluate the effectiveness of different visualization methods.
- Clean and prepare data for analysis.

0.3 INSTRUCTIONS

- Restart the kernel and run the whole notebook again before you submit.
- As much as possible, try and stick to the hints and functions we import at the top of the homework, as those are the ideas and tools the class supports and is aiming to teach. And if a problem specifies a particular library you're required to use that library, and possibly others from the import list.
- Please use .head() when viewing data.

0.4 About

This exercise relates to the College data set, which can be found in the file College.csv. It contains a number of variables for 777 different universities and colleges in the US. The variables are

- `Private`: Public/private indicator
- `Apps`: Number of applications received
- `Accept`: Number of applicants accepted
- `Enroll`: Number of new students enrolled
- `Top 10 percent`: New students from top10% of high school class
- `Top 25 percent`: New students from top 25% of high school class
- `F.Undergrad`: Number of full-time. undergraduates
- `P.Undergrad`: Number of part-time undergraduates
- `Outstate`: Out-of-state tuition
- `Room.Board`: Room and board costs

- `Books`: Estimated book costs
- `Personal`: Estimated personal spending
- `PhD`: Percent of faculty with Ph.D.'s
- `Terminal`: Percent of faculty with terminal degree • S.F.Ratio: Student/faculty ratio
- `perc alumni`: Percent of alumni who donate
- `expend`: Instructional expenditure per student
- `grad.Rate`: Graduation rate

[ ]:
```
""" RUN THIS CELL TO GET THE RIGHT FORMATTING """
import requests
from IPython.core.display import HTML
css_file = 'https://raw.githubusercontent.com/bsethwalker/clemson-cs4300/main/
 ↪css/cpsc6300.css'
styles = requests.get(css_file).text
HTML(styles)
```

[ ]:
```
import pandas as pd
import numpy as np
from pandas.plotting import scatter_matrix
from matplotcheck.base import PlotTester
from matplotlib.patches import PathPatch
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
%matplotlib inline
```

Exercise 1: Load Data set

- Using Pandas, load the `College.csv` file from 'data/College.csv' into a DataFrame named `college`.
- Take a look at the loaded data.
- Rename the first column as `Name` and display the first few rows of the DataFrame.

[ ]:
```
"""Write your code for exercise-1 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Notice here that the table that was output through the jupyter notebook has additional formatting rendered with it. When we ask in this class to "report" a dataframe, know that the last line of a code cell be the variable you are reporting will give this nice formatting.

[ ]:
```
# this will render good (enough) looking formatting for the data
college
```

[ ]:
```
# this will print plain text
print(college)
```

Exercise 2: Elite Institutions

- Create a new qualitative variable, called `Elite`, by binning the Top 10 percent variable.
- Here you are going to divide universities into two groups based on whether or not the proportion of students coming from the top 10% of their high school classes exceeds 50%.
  - Categorize `Elite` column into `Yes` or `No`.(i.e `Elite` column should only contain `Yes` or `No` values.)
  - Value for `Elite` column shoud be `Yes` if the proportion of students coming from the top 10% of their high school classes is greater than 50% (i.e top10perc > 50)
  - Else `Elite` column should be `No`

[ ]:
```
"""Write your code for exercise-2 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 3: Acceptance Rates

- Create a new column called `AcceptRate` that contains the acceptance rate for each university.
- Calculate aaceptance rate using following formula:
  - $(Accept/Apps) * 100$

[ ]:
```
"""Write your code for exercise-3 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 4:

- How many elite schools are there?
- Extract and store number of elite schools form our dataset to variable named `num_elite_schools`.

[ ]:
```
"""Write your code for exercise-4 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 5: Acceptance Rate Comparison

- Create a boxplot comparing the acceptance rates of elite and non-elite universities.

1. To create the box plot, you need to set up the Figure and Axes objects using `plt.subplots()`.
   - These two objects (fig and ax) will allow you to control various properties of the figure and plot.
   - `fig` is the Figure object: It serves as the overall container for the plot.
   - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc.
     `Example code: fig, ax = plt.subplots(figsize=(10, 6))`
2. Customize the Plot:
   - You need to set the title of the plot, the labels for the x-axis and y-axis, and format the ticks on the x-axis for better readability.

[ ]:
```
"""Write your code for exercise-5 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 6: Cost Comparisons

- Create two side-by-side histograms (using subplots) showing the distribution of out of state tuition for elite and non-elite institutions.

1. To create plot, you need to set up the Figure and Axes objects using `plt.subplots()`.
   - These two objects (fig and ax) will allow you to control various properties of the figure and plot.
   - `fig` is the Figure object: It serves as the overall container for the plot.
   - `ax` is the Axes object: This is where the actual data points will be plotted, including x and y axes, labels, etc.
     `Example code: fig, ax = plt.subplots(1, 2, figsize=(10, 6))`
   - Here `plt.subplots(1, 2)` creates one row and two columns of subplots. This results in two individual Axes that will be side by side.
2. Customize the Plot:
   - You need to set the title of the plot, the labels for the x-axis and y-axis, and format the ticks on the x-axis for better readability.

[ ]:
```
"""Write your code for exercise-6 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 7:

- Which University has the most students in the top 10% of class?
- Extract the name of University that has the most students in the top 10% of class and store in new variable named `top_university`.

[ ]:
```
"""Write your code for exercise-7 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 8:

- Which university has the smallest acceptance rate?
- Extract the name of University that has the smallest acceptance rate and store in new variable named `university_smallest_accept_rate`.

[ ]:
```
"""Write your code for exercise-8 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 9:

- Which university has the most liberal acceptance rate?
- Extract the name of University that has the most liberal acceptance rate and store in new variable named `university_most_liberal_accept_rate`.

[ ]:
```
"""Write your code for exercise-9 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 10:

- Calculate correlation between out-of-state tuition and graduation rate and store it to variable named `correlation`.
- Refer to this document pandas.DataFrame.corr on how to calculate correlation value using pandas .corr().

[ ]:
```
"""Write your code for exercise-10 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

Exercise 11:

- Calculate Clemson University's acceptance rate and store it to variable named `clemson_accept_rate`.

[ ]:
```
"""Write your code for exercise-11 here:"""

# your code here
raise NotImplementedError
```

[ ]:
```

```

1 END